

Desarrollo Aplicaciones Web - Asistido con IA

Esta es una hoja de ruta: un intento de describir, paso a paso, todo el proceso de construcción de una aplicación web (un software), de principio a fin, desde una computadora recién empezada hasta la aplicación terminada y publicada en internet. La idea es que cada paso pueda seguirse en orden, sin ambigüedad, sin dejar nada librado a la interpretación de quien lo lee.

A lo largo de esta guía, cada paso está marcado según con qué frecuencia se repite. Estos tres colores identifican cada caso, y cada paso del documento va a aclarar además, en palabras, a cuál de los tres corresponde:

-  **por única vez** — se hace una sola vez en toda la vida de la PC, y no se repite nunca más
-  **por proyecto** — se repite cada vez que se arma una aplicación nueva
-  **por cada iteración** — se repite en cada sesión de trabajo sobre una misma aplicación

Conceptos previos (leer antes de empezar)

¿Qué es “código”? Es el texto escrito en un lenguaje que la computadora puede entender y ejecutar. Es lo que hace que la aplicación funcione — sin código, no hay aplicación. Analogía: pensalo como la receta de un plato de cocina. La receta está escrita en un idioma que un cocinero entiende y sabe ejecutar paso a paso para producir el plato final. El código es igual, solo que quien “cocina” siguiendo esas instrucciones es la computadora. En este curso, esa receta no la va a escribir el alumno a mano: se la va a pedir a la IA (OpenCode), que la escribe por él. Pero el resultado sigue siendo código igual, aunque nadie lo haya tipeado directamente.

¿Qué es un “programa”, una “aplicación”, una “App” o un “software”? Son distintas formas de llamar exactamente a la misma cosa: el conjunto completo de código que, todo junto, hace algo útil (por ejemplo, guardar y mostrar una lista de clientes). “Software” y “programa” son los términos más formales o técnicos; “aplicación” y “App” son los que se usan en el habla cotidiana (como cuando se dice “bajate la App del banco”). En este documento se usan todos como sinónimos, indistintamente.

¿Qué es una “versión”? Es una “foto” de la aplicación (App) en un momento determinado del tiempo — no del código en abstracto, sino de cómo se ve y se comporta la aplicación en ese momento puntual. Ejemplo: la primera vez que se construye la aplicación, tiene el fondo blanco. Se guarda ese avance: esa es la versión 1. No termina de convencer, así que se le pide a la IA “usá colores más atractivos, y que el fondo sea color crema”. Ese cambio se guarda como la versión 2 — una foto distinta, más nueva, de la misma aplicación. Sirve para dos cosas: poder volver atrás si algo se rompe, y poder saber exactamente qué cambió y en qué momento.

¿Qué es un “historial de versiones”? Es la colección completa y ordenada de todas esas “fotos” guardadas a lo largo del tiempo, una después de la otra, como un álbum de fotos que muestra la evolución completa de un proyecto desde el primer día hasta hoy.

¿Qué es “la nube”? Son, ni más ni menos, computadoras que pertenecen a otras empresas (por ejemplo Microsoft, en el caso de GitHub; otras empresas, en el caso de Supabase y Vercel) a las

que se accede a través de internet. Analogía: es como alquilar un espacio en un guardamuebles de otra empresa, en vez de guardar todo únicamente en el placard de casa. La ventaja es que si algo le pasa a la propia casa (la PC se rompe, se pierde, se roba), lo guardado en el guardamuebles sigue estando disponible. Y además, otras personas o servicios autorizados pueden acceder a ese mismo guardamuebles sin tener que ir hasta la casa de uno.

¿Qué es un “servidor”? Es una computadora que está encendida y lista para responder pedidos en cualquier momento. Analogía: es como un mozo de restaurante que nunca se va, siempre disponible para atender cualquier pedido que llegue, a cualquier hora. Puede ser de dos tipos: Público (en internet, accesible por cualquiera con un link — así corre la aplicación una vez publicada, en Vercel) o Privado/local (corriendo únicamente dentro de la propia PC, donde nadie de afuera puede entrar — así corre la aplicación mientras todavía se está construyendo, en modo desarrollo). Es el mismo concepto de fondo en los dos casos (una máquina lista para atender pedidos); lo único que cambia es quién tiene permiso para acceder a ella.

¿Qué es la “terminal”? Es una ventana de la computadora donde, en vez de hacer clic con el mouse sobre íconos y botones, se escriben instrucciones de texto que la computadora ejecuta al instante, apenas se presiona Enter. Analogía: si el uso normal de una computadora (con el mouse) es como manejar un auto automático, la terminal es como manejar uno con caja manual: hay que darle instrucciones más precisas, pero a cambio se puede hacer exactamente lo que se necesita, sin depender de que exista un botón para eso. En este curso se usa muy poco, y solo para un puñado de cosas fijas y repetidas (esto se aclara en cada paso donde aparece). Se puede abrir como ventana propia de Windows, o usar la terminal integrada dentro de VS Code — es exactamente la misma terminal, solo cambia desde dónde se la abre.

 Un detalle importante sobre la terminal: cada comando de este documento tiene que escribirse en el lugar correcto. Hay una diferencia entre escribirle algo a Windows (en una terminal, antes de abrir OpenCode, o en una terminal nueva y separada mientras OpenCode ya está abierto en otra ventana) y escribirle algo al agente (dentro de una sesión de OpenCode ya abierta, en su propio cuadro de texto). Un comando de terminal (como `opencode mcp auth supabase`) nunca se le escribe al agente dentro de su chat — y un pedido en lenguaje natural para el agente nunca se escribe en una terminal común. Cada paso de esta guía aclara en cuál de los dos lugares corresponde escribir.

¿Qué es la Inteligencia Artificial (IA), un “modelo” y un “proveedor de IA”? La Inteligencia Artificial, en el contexto de este curso, es un programa capaz de entender pedidos escritos en lenguaje natural — como si se le hablara a otra persona — y responder generando código como resultado. Ese programa de IA en particular se llama modelo (es su nombre técnico). Distintas empresas fabrican y ofrecen sus propios modelos — a esas empresas se las llama proveedores (por ejemplo, Anthropic, que fabrica los modelos de la familia Claude). OpenCode es la herramienta instalada en la PC que permite hablarle a esos modelos desde la terminal, como si fuera un intermediario entre el alumno y la IA.

¿Qué es un “prompt”? Es el pedido concreto que se le escribe a la IA en lenguaje natural — por ejemplo: “Creá un formulario con estos campos: nombre, apellido y correo”. Analogía: es exactamente como pedirle algo a un empleado nuevo, capaz pero que no puede leer la mente: cuanto más clara y detallada sea la instrucción, mejor va a ser el resultado. Un pedido vago

(“hacé algo lindo”) da resultados impredecibles; un pedido preciso (“un botón azul, arriba a la derecha, que diga Guardar”) da resultados confiables.

¿Qué son las “capas” de una aplicación? Toda aplicación moderna se puede pensar dividida en grandes partes, cada una con una responsabilidad bien distinta a las demás: Frontend (lo que el usuario ve y toca en la pantalla: botones, textos, colores, formularios), Aplicación o Backend (lo que procesa la lógica por detrás y decide qué hacer con cada acción del usuario), Datos (el lugar donde se guarda la información de forma permanente) e Infraestructura (dónde vive físicamente todo esto, para que otras personas puedan acceder: servidores, publicación). Analogía: pensalo como un restaurante. El Frontend es el salón, la carta y los mozos — lo que el cliente ve y con lo que interactúa. La Aplicación/Backend es la cocina, donde se procesan los pedidos y se decide cómo prepararlos. Los Datos son la despensa, donde están guardados los ingredientes (la información) que la cocina necesita para trabajar. La Infraestructura es el edificio entero: el lugar físico que hace posible que todo lo anterior exista y funcione junto. Este orden de capas se va a repetir constantemente a lo largo del curso.

¿Qué es una “organización” (en Supabase)? Es el nivel más alto dentro de una cuenta de Supabase: un espacio general que puede contener varios proyectos (varias bases de datos) adentro, uno al lado del otro. Para este curso, alcanza y sobra con la organización que Supabase crea automáticamente por defecto la primera vez — no hace falta crear una organización nueva a mano.

¿Qué es un “dashboard”? Es la pantalla principal de un servicio (Supabase, Vercel, GitHub) que aparece apenas se inicia sesión, y desde donde se accede a todas las opciones y proyectos disponibles de esa cuenta. Es, en cierto sentido, el “escritorio” de cada servicio.

¿Qué es una “API”? Es el punto de conexión que un servicio deja disponible para que otros programas puedan hablarle y pedirle cosas. Cuando en Supabase se dice “ir a Configuración → API”, ahí es exactamente donde están los datos (Project URL, Publishable key) que la aplicación va a necesitar para poder conectarse y conversar con esa base de datos.

¿Qué es “Tailwind”? Es el sistema con el que se escriben los estilos visuales de la aplicación (colores, tamaños, espaciados entre elementos) dentro de Next.js. El alumno no necesita escribir ninguna línea de Tailwind a mano — se lo pide a OpenCode en lenguaje natural (“que el botón sea verde y más grande”) y la herramienta se encarga de traducir eso a código por detrás.

¿Qué significa “publicar” una aplicación (también llamado “Deploy” o “desplegar”)? Es el momento exacto en el que la aplicación deja de vivir únicamente en la computadora de quien la construyó, y pasa a vivir en un servidor público en internet, accesible por cualquier persona, desde cualquier lugar, en cualquier momento — sin que la PC original necesite estar prendida ni conectada. “Deploy” y “desplegar” son sinónimos técnicos, en inglés y en español, de la misma palabra: “publicar”.

¿Qué es Vercel? Es el servicio elegido en este curso para publicar la aplicación. Toma el código que está guardado en el repositorio de GitHub y lo pone a correr en un servidor público de internet, generando como resultado el link final que cualquier persona puede abrir desde su navegador, en su celular o en su computadora.

¿Qué significa “guardar cambios en GitHub” (commit + push)? Es el proceso técnico de tomar el estado actual del código y agregarlo como una versión nueva al historial del repositorio, para que quede guardado en la nube. “Commit” es el paso de armar esa “foto” del momento actual; “push” es el paso de enviar esa foto hacia la nube, a GitHub. En este curso, ambos pasos se le piden juntos a OpenCode con un solo pedido en lenguaje natural, sin necesidad de escribir los comandos técnicos por separado.

¿Qué es una “base de datos”? Es el lugar donde una aplicación guarda la información que necesita recordar de forma permanente (por ejemplo, los datos de cada cliente que se va cargando). Sin base de datos, la aplicación “olvidaría” todo apenas se cierra o se reinicia.

¿Qué es un “repositorio”? Es una carpeta especial, guardada en la nube (específicamente en GitHub), que contiene el código de un proyecto puntual junto con todo su historial de versiones completo. Es el lugar común donde se sube cada avance para que todos los fragmentos del proyecto queden unidos y ordenados en un solo lugar.

¿Qué significa que una aplicación esté “corriendo”? Significa que en ese momento está ejecutándose activamente, funcionando y lista para responder — ya sea de forma privada en la PC (modo desarrollo) o de forma pública en un servidor (producción).

¿Qué es un “stack”? Es el conjunto completo de tecnologías elegidas para construir una aplicación, todas usadas juntas, como piezas complementarias entre sí — es decir, con qué herramientas puntuales se va a construir el software de este curso. Analogía: es como la lista de ingredientes elegidos para preparar una receta particular — no cualquier combinación funciona bien junta, pero una vez elegida una combinación que funciona, se puede repetir una y otra vez con resultados consistentes. El stack de este curso es: Next.js (la estructura base sobre la que se construye) + Supabase (base de datos) + Vercel (publicación) + OpenCode (agente de IA que construye todo lo anterior a pedido).

¿Qué es “modo desarrollo” y qué es “producción”? Son los dos grandes momentos de la vida de una aplicación: Modo desarrollo (la aplicación corre únicamente en la PC de quien la está construyendo, en un servidor privado y local, mientras todavía se está probando y modificando — nadie más puede verla ni acceder a ella) y Producción (la aplicación ya está publicada, corriendo en un servidor público —Vercel—, accesible por cualquier persona desde un link, en cualquier momento). Es exactamente la misma aplicación en los dos casos — lo que cambia es el momento de su vida en el que se encuentra, y quién puede acceder a ella.

¿Qué es la “PC de desarrollo”? Es la computadora específica donde la aplicación se construye, se prueba y se modifica, mientras todavía no está terminada — la computadora donde corre en modo desarrollo, antes de mudarse a producción.

¿Qué es “MCP”? Son las siglas de “Model Context Protocol”. Es un mecanismo estandarizado que le permite a una IA (en este caso, OpenCode) conectarse directamente con un servicio externo (en este caso, Supabase) y operar sobre él con permisos acotados a un uso puntual — por ejemplo, crear o modificar tablas — sin necesidad de manejar una clave de acceso genérica a toda la cuenta. Analogía: es como darle a un empleado de mantenimiento una llave que abre únicamente la puerta del depósito que tiene que arreglar, en vez de darle el llavero maestro de todo el edificio.

¿Qué es “OAuth”? Es el mecanismo que permite autorizar a una herramienta (como OpenCode) a usar una cuenta (como la de Supabase) sin tener que escribir ni compartir ninguna contraseña con esa herramienta. En la práctica, se ve como una ventana del navegador que se abre sola, donde se inicia sesión y se hace clic en “Autorizar” — nada más. Analogía: es como mostrar una credencial de acceso en la entrada de un edificio, en vez de entregarle a un visitante la llave de tu casa. El visitante (OpenCode) queda habilitado a entrar por la puerta que le corresponde, sin que nadie tenga que darle una copia de la llave maestra.

Resumen del stack de este curso, con analogía

Capa	Herramienta	Analogía (Toy Story 5)
Agente de IA / codificación	OpenCode	El equipo de animadores y técnicos que efectivamente construye cada escena
Frontend	Next.js (JavaScript + Tailwind)	Lo que el espectador ve en pantalla: personajes, escenarios, movimiento
Base de datos	Supabase (conectada vía MCP)	La memoria interna de la historia: quién es cada personaje, qué le pasó, qué relación tiene con los demás
Despliegue	Vercel	El estreno: la película sale de la sala de edición y llega a los cines/streaming para que cualquiera la vea

El panorama general: ¿cómo es el proceso de construir un software?

Construir un software (una aplicación web) es, en esencia, como construir una película animada, 100% digital — como Toy Story 5. En una película así, nadie graba una toma real con una cámara: los personajes, los escenarios, los fondos, cada escena, todo se describe con palabras, y a partir de esa descripción la película se genera. Es exactamente lo que va a pasar en este curso: en vez de escribir código a mano, se va a describir con palabras lo que se necesita, y la IA lo va a generar.

Ahora bien: una película como esa no la hace una sola persona. Se conforma un equipo de trabajo, donde se dividen tareas y responsabilidades: un grupo se encarga de las escenas 1, 2 y 3; otro grupo, en paralelo, trabaja sobre escenas completamente distintas. Cada equipo avanza con su propio fragmento de la historia, por su lado.

Ahí aparece la pregunta clave: ¿cómo hacen para que todos esos fragmentos, contruidos por separado, se terminen uniendo en una sola historia coherente? Necesitan un lugar común donde cada uno pueda subir lo que fue avanzando, y donde quede guardado un registro ordenado de cada aporte, a medida que se va sumando. Ese lugar común es exactamente lo que es un repositorio en la nube (GitHub): el sitio donde todos los que están trabajando en el proyecto suben sus cambios, y donde cada avance que se sube queda guardado como una versión nueva.

Con esa idea ya clara, el proceso completo, de principio a fin, queda así:

1. Se piensa el objetivo de la aplicación: qué se va a hacer, para quién está dirigida, y cuáles son sus límites y alcances — recién a partir de ese objetivo se puede pensar qué información va a guardar y qué pantallas va a tener. Es como escribir el guión de la película antes de empezar a describir ninguna escena.
2. Se prepara el terreno: las cuentas y herramientas que van a hacer falta durante todo el curso. Se hace una única vez, como preparar todo el software y las cuentas del estudio de animación antes de empezar a producir. En orden: cuenta de GitHub, VS Code, Terminal, Node.js, Git, Git configurado y vinculado a GitHub, OpenCode instalado, modelo de IA elegido dentro de OpenCode, plugin de Supabase, cuenta de Supabase, cuenta de Vercel.
3. Se arma el “esqueleto” de la aplicación: la estructura base (con Next.js), como generar el primer armado crudo de la película: las escenas ya existen, en orden, aunque todavía sin ningún retoque.
4. Se prepara dónde se va a guardar la información (la base de datos, con Supabase) y se la conecta con el esqueleto. Es como el guión técnico que dice qué pasa en cada escena: no siempre se ve a simple vista en pantalla, pero sin eso la historia no tiene sentido.
5. Se construyen las pantallas que el usuario va a ver y usar. Es la etapa de retoques: se prueban distintos colores, distintos fondos, se ajusta lo que no queda bien, hasta lograr el resultado final.
6. Se prueba que todo funcione como se espera, mientras la aplicación todavía está solo en la computadora de quien la construye (a esto se le llama “modo desarrollo”). Es como una proyección privada, solo para el equipo, antes del estreno: se revisa la película entera y se corrige lo que haga falta, mientras nadie de afuera la vio todavía.
7. Se publica: la aplicación deja de vivir únicamente en una computadora, y pasa a vivir en internet, accesible por cualquiera desde un link (a esto se le llama “producción”, y el servicio que lo hace posible es Vercel). Es el estreno: la película sube a la plataforma de streaming, y a partir de ese momento cualquier persona, desde cualquier lugar, puede verla.

Una diferencia clave entre la película y la aplicación: una vez que la película está terminada y publicada, queda cerrada — los espectadores la ven, la consumen, pero no guardan ningún dato ahí. Una aplicación es distinta: los usuarios interactúan con ella y generan información propia (cargan un cliente, registran un dato, guardan algo), y esa información tiene que quedar guardada en algún lugar, de forma permanente, incluso después de que el usuario cierre la aplicación. Ahí es donde entra Supabase: es el servicio en la nube que le da a la aplicación un lugar donde guardar, de forma permanente, todo lo que sus usuarios van registrando.

Todo lo que sigue en este documento es el detalle, paso a paso, de estas 7 etapas.

PARTE 1 — Instalación y configuración de herramientas necesarias y entorno

1. Abrir la Terminal de Windows — por única vez

No hace falta instalar absolutamente nada para este paso: Windows ya trae una terminal incluida de fábrica. Es una de las ventanas que se van a usar durante el curso para hablar con OpenCode.

- 1.1) Buscar “Terminal” (o “Símbolo del sistema”) en el menú de inicio de Windows
- 1.2) Abrirla

2. Instalar VS Code — por única vez

VS Code es el programa donde vamos a ver y organizar los archivos del proyecto a lo largo del curso. Trae, además, una terminal integrada propia (exactamente la misma terminal de Windows, pero abierta desde adentro del programa), que es la ventana donde en la práctica se va a hablar más seguido con OpenCode.

- 2.1) Ir a <https://code.visualstudio.com/download>
- 2.2) Descargar el instalador para Windows
- 2.3) Ejecutarlo, aceptar todo con “Next” y “Finish”

3. Instalar Git — por única vez

Git es el programa que corre localmente en la propia PC y que sabe “hablar” con GitHub: es el encargado de empaquetar los cambios del código y enviarlos hacia la nube. Para no confundirlos: GitHub es el servicio en la nube (la “caja fuerte” en internet); Git es la herramienta instalada en la PC que se comunica con esa caja fuerte.

- 3.1) Ir a <https://git-scm.com/downloads>
- 3.2) Descargar el instalador para Windows
- 3.3) Ejecutar, dejando todas las opciones por defecto, “Next” hasta el final

4. Crear cuenta en GitHub — por única vez

GitHub guarda el historial de versiones del código, y es además el lugar desde donde Vercel va a leer el proyecto para poder publicarlo más adelante.

- 4.1) Ir a <https://github.com/join>
- 4.2) Completar correo, contraseña y nombre de usuario
- 4.3) Verificar el correo

5. Configurar Git en la PC, vinculado a GitHub — por única vez

Antes de poder enviar cambios a GitHub, Git necesita dos cosas: saber “quién sos” (para dejarlo registrado en cada versión guardada) y tener el permiso concreto de esa cuenta de GitHub para poder enviarle cosas.

- 5.1) Abrir la Terminal de Windows (o la terminal integrada de VS Code)
- 5.2) Copiar y pegar en la consola (reemplazando “Nombre Apellido” por el nombre real):
`git config --global user.name "Nombre Apellido"`
- 5.3) Copiar y pegar en la consola (reemplazando el correo por el correo real, el mismo con el que se creó la cuenta de GitHub): `git config --global user.email "correo@ejemplo.com"`
- 5.4) Hacer una prueba de conexión (por ejemplo, el primer envío de cambios a GitHub del punto 22) — en ese momento se va a abrir automáticamente una ventana del navegador, pidiendo iniciar sesión en GitHub

- 5.5) Iniciar sesión ahí y autorizar el acceso (esto queda guardado para siempre en esa PC; no hace falta repetirlo en cada sesión)

6. Instalar Node.js — por única vez

Node.js es el motor que necesitan tanto Next.js como OpenCode para poder funcionar. Es infraestructura de base pura: nunca se va a usar directamente ni se le va a “hablar”, pero sin él, nada de lo demás arranca — como el motor de un auto, que no se toca directamente pero sin el cual el auto no se mueve.

- 6.1) Ir a <https://nodejs.org>
- 6.2) Descargar la versión LTS (siglas de “Long Term Support”, la versión más estable y recomendada)
- 6.3) Ejecutar el instalador, “Next” hasta el final
- 6.4) Verificar que quedó instalado. Copiar y pegar en la consola: `node -v` (debería aparecer un número de versión en pantalla)

7. Instalar OpenCode — por única vez

OpenCode es el agente de inteligencia artificial que va a escribir el código por nosotros, a partir de lo que se le pida en lenguaje natural. Es, sin exagerar, la pieza central de todo el curso: en vez de escribir cada línea de código a mano, se la describimos y él la construye.

- 7.1) Abrir la Terminal de Windows
- 7.2) Copiar y pegar en la consola, tal cual, en una sola línea: `npm install -g --allow-scripts=opencode-ai opencode-ai`
- 7.3) Verificar que quedó instalado. Copiar y pegar en la consola: `opencode --version` (debería devolver un número de versión, sin ninguna ventana de error)

 Este comando es el único que hay que usar para instalar OpenCode. Existe una versión más corta (`npm install -g opencode-ai`, sin el fragmento `--allow-scripts`) que también parece funcionar, pero deja el programa a medio instalar: Windows bloquea, por seguridad, un paso interno de la instalación, y como consecuencia el programa queda roto (puede aparecer más adelante un error como “aplicación de 16 bits no compatible” al intentar usarlo). Si esto llegara a pasar, ver la sección “Solución de problemas” al final de este documento.

8. Elegir el modelo de IA que va a usar OpenCode — por única vez

OpenCode ya viene con modelos gratuitos disponibles por dentro (a través de “OpenCode Zen”), sin necesidad de crear ninguna cuenta nueva, sin tarjeta de crédito, y sin ningún proceso de autorización.

- 8.1) Copiar y pegar en la consola: `opencode`
- 8.2) Dentro de OpenCode, copiar y pegar: `/models`
- 8.3) Elegir, de la lista, un modelo marcado como “Free” (el recomendado para el curso: DeepSeek V4 Flash Free)
- 8.4) Cuando pregunte el nivel de esfuerzo (default / high / max), elegir default

⚠ Usamos DeepSeek V4 Flash Free porque no requiere un login externo complicado — otras opciones (como GitHub Copilot dentro de OpenCode) pueden entrar en loops de autenticación que traban la clase.

9. Instalar el plugin de Supabase para OpenCode 🔵 — *por única vez*

Este plugin le enseña a OpenCode a hablar específicamente con Supabase: sin él, OpenCode no tiene forma de conectarse a una base de datos de Supabase ni de gestionar su estructura. Es un paso de instalación, parecido al del propio OpenCode — se hace una sola vez por PC.

- 9.1) Cerrar cualquier sesión de OpenCode que esté abierta (este comando se escribe en una terminal común, nunca dentro del chat de OpenCode)
- 9.2) En la Terminal de Windows, copiar y pegar: `opencode plugin opencode-supabase`
- 9.3) Esperar a que la consola muestre “Installed opencode-supabase”

10. Crear cuenta en Supabase 🔵 — *por única vez*

Supabase es el servicio que va a alojar la base de datos de cada aplicación construida durante el curso: todo lo que la aplicación necesite recordar va a vivir ahí.

- 10.1) Ir a <https://supabase.com>
- 10.2) Click en “Start your project” o “Sign in”
- 10.3) Elegir “Continue with GitHub” (así se reutiliza la misma identidad de GitHub, sin necesidad de crear ni recordar una contraseña más)

⚠ Usá esta cuenta de Supabase únicamente para practicar en este curso. Nunca uses esta misma cuenta para un proyecto real con datos de un cliente.

11. Crear cuenta en Vercel 🔵 — *por única vez*

Vercel es el servicio que va a publicar la aplicación en internet, generando el link con el que cualquiera va a poder acceder a ella.

- 11.1) Ir a <https://vercel.com>
- 11.2) Click en “Sign Up”
- 11.3) Elegir “Continue with GitHub”

PARTE 2 — Construcción del software

12. Crear repositorio en GitHub 🟡 — *por proyecto*

El repositorio guarda el código del proyecto puntual que se está construyendo. Es muy importante no confundir dos cosas que suenan parecido pero no son lo mismo: el repositorio (GitHub) guarda el código, es decir, el programa en sí; el “proyecto” de Supabase (que se ve en el punto 14) es la base de datos, es decir, la información que ese programa usa. Uno es la receta; el otro es la despensa. Para que esto quede siempre claro a simple vista, en esta guía se usan nombres distintos y reconocibles para cada uno: la aplicación se llama `appclientes`, y su base de datos se llama `datos_appclientes`.

- 12.1) Ir a <https://github.com/new>
- 12.2) Poner como nombre del repositorio: appclientes
- 12.3) Dejarlo en “Private” o “Public”, según se prefiera
- 12.4) Click en “Create repository”

13. Clonar el repositorio a la PC 🟡 — *por proyecto*

“Clonar” significa traer una copia del repositorio (todavía vacío) desde GitHub hacia la propia PC, creando la carpeta local ya conectada a GitHub desde el primer momento — en un solo paso, sin necesidad de crear una carpeta vacía por un lado y conectarla por otro más adelante.

- 13.1) En GitHub, entrar al repositorio creado en el paso anterior y copiar su link (botón verde “Code” → pestaña “HTTPS” → copiar la URL, algo como <https://github.com/usuario/appclientes.git>)
- 13.2) Abrir la Terminal de Windows
- 13.3) Posicionarse en la carpeta donde se guardan habitualmente los proyectos (escribiendo `cd` seguido de la ruta, o pegando esa ruta desde el explorador de archivos)
- 13.4) Copiar y pegar en la consola (reemplazando por el link real copiado en 13.1): `git clone https://github.com/usuario/appclientes.git`
- 13.5) Si es la primera vez que se hace esto en la PC, va a aparecer la ventana de login de GitHub descrita en el punto 5.4 — autorizar ahí
- 13.6) Verificar: ya debería existir la carpeta del proyecto, con el mismo nombre del repositorio, conectada a GitHub desde el día uno

14. Crear el proyecto (= base de datos) en Supabase 🟡 — *por proyecto*

En Supabase, crear un “proyecto” es exactamente lo mismo que crear la base de datos — no son dos pasos separados, son la misma acción. Esto genera automáticamente tres datos propios y exclusivos de ese proyecto puntual: el Password (contraseña técnica avanzada, casi no se usa directamente en este curso), el Project URL y la Publishable key (estas dos últimas sí se usan, y mucho). Se hace en este momento, antes de construir la aplicación, para tener todo listo y poder pedirle el trabajo completo a OpenCode en un solo prompt.

- 14.1) Ir a <https://supabase.com/dashboard> e iniciar sesión (si no lo está ya)
- 14.2) Click en “New Project”
- 14.3) Elegir organización (o aceptar la que Supabase propone por defecto)
- 14.4) Poner como nombre del proyecto: datos_appclientes
- 14.5) Crear y guardar en un lugar seguro la contraseña del proyecto
- 14.6) Elegir la región geográfica más cercana
- 14.7) Click en “Create new project”
- 14.8) Esperar entre 1 y 2 minutos de aprovisionamiento (el tiempo que tarda Supabase en dejar la base de datos completamente lista para usarse)

15. Copiar Project URL y Publishable key 🟡 — *por proyecto*

Son los dos datos que la aplicación va a usar siempre, para leer y escribir información en la base de datos, una vez que ya esté todo construido.

- 15.1) Dentro del proyecto de Supabase, ir a Configuración (ícono de engranaje) — API
- 15.2) Copiar el valor de “Project URL”
- 15.3) Copiar el valor de “Publishable key”

16. Conectar OpenCode con este proyecto de Supabase por MCP — *por proyecto*

Este es el paso más delicado de todo el documento, y el que más conviene seguir sin saltar ningún sub-paso. El objetivo es que OpenCode quede autorizado a crear y modificar tablas dentro de este proyecto puntual de Supabase, sin manejar nunca una contraseña de base de datos.

Parte A — Conectar la cuenta

- 16.1) Dentro de la carpeta del proyecto ya clonado, abrir la Terminal de Windows
- 16.2) Copiar y pegar en la consola: `opencode`
- 16.3) Ya dentro de la sesión de OpenCode (en su cuadro de texto, no en la terminal), escribir: `/supabase`
- 16.4) Va a aparecer un botón o instrucción para conectar la cuenta — seguirlo. Se abre el navegador
- 16.5) Iniciar sesión con la cuenta de Supabase y autorizar (“Authorization Successful”)
- 16.6) Cerrar esa pestaña del navegador y volver a OpenCode

Parte B — Elegir el proyecto y pedir la conexión MCP

- 16.7) Dentro de la sesión de OpenCode, escribir: `List my Supabase projects` (debería aparecer `datos_appclientes`, con su estado `ACTIVE_HEALTHY`)
- 16.8) Escribir: `Conectá el MCP de Supabase para el proyecto datos_appclientes`
- 16.9) Se va a abrir el navegador en Supabase Studio, en una pantalla de configuración de conexión

Parte C — Configurar los permisos en Supabase Studio

- 16.10) En esa pantalla, confirmar que el switch “Read-only” esté DESACTIVADO (en gris) — si estuviera activado, OpenCode solo podría leer la base de datos, no crear ni modificar tablas
- 16.11) En “Feature groups”, confirmar que esté incluido “Database” (viene así por defecto)
- 16.12) Copiar el bloque de configuración (JSON) que aparece en el paso 1 de esa misma pantalla (“Configure MCP”)

Parte D — Aplicar la configuración y autenticar el MCP

- 16.13) Volver a OpenCode y pegar ese bloque donde el agente lo esté pidiendo
- 16.14) OpenCode va a confirmar que guardó la configuración, y va a pedir 4 cosas en este orden: cerrar OpenCode, ejecutar un comando en la terminal, completar un login por navegador, y volver a abrir OpenCode
- 16.15) Cerrar la sesión actual de OpenCode

- 16.16) En la Terminal de Windows (fuera de OpenCode), copiar y pegar: `opencode mcp auth supabase`
- 16.17) Se abre el navegador — completar el inicio de sesión con la cuenta de Supabase (usando el mismo perfil del navegador donde ya se inició sesión antes)
- 16.18) Volver a la terminal, copiar y pegar: `opencode`
- 16.19) Confirmar en el panel de MCP (dentro de OpenCode) que ya no diga “Needs auth” junto a “supabase”

⚠ Este es el paso más importante de todo el capítulo. En vez de copiar una clave secreta de toda la cuenta a mano, OpenCode queda conectado directo a Supabase con un permiso limitado a este proyecto puntual, con autenticación por navegador en dos niveles (la cuenta, y el MCP del proyecto). Todo cambio de estructura de datos que se pida de acá en más queda registrado como una migración versionada — si algo sale mal, se puede rastrear y revertir.

17. Pedirle a OpenCode el prompt técnico inicial 🟡 — *por proyecto*

Antes de pedirle nada sobre el negocio puntual de la aplicación (qué información va a guardar, qué pantallas va a tener), conviene dejar asentada la configuración técnica del proyecto, siempre igual, sin importar qué aplicación se vaya a construir. Esto separa lo técnico (fijo) de la lógica de negocio (distinta en cada proyecto), y evita que el alumno tenga que “inventar” cómo pedir la parte técnica.

- 17.1) Estando ya dentro de la sesión de OpenCode (en la carpeta del proyecto), copiar y pegar, reemplazando los corchetes por los datos reales copiados en el punto 15:

Quiero crear una aplicación web nueva en esta carpeta, usando Next.js con JavaScript y Tailwind. La base de datos va a ser Supabase, y quiero que gestiones la estructura de tablas usando el MCP de Supabase que ya está conectado — no uses una cadena de conexión directa con contraseña.

Configurá el proyecto para que la aplicación se conecte a Supabase usando estas credenciales:
 Project URL: [pegar acá]
 Publishable key: [pegar acá]

Guardá esas dos credenciales en el archivo `.env.local` del proyecto, para que de ahora en adelante ya las tengas disponibles vos mismo, sin que te las tenga que volver a pasar.

No crees todavía ninguna tabla ni pantalla — quiero primero confirmar que el proyecto base quedó armado correctamente.

- 17.2) Esperar a que OpenCode termine de armar la estructura base
- 17.3) Confirmar que no haya errores antes de seguir al paso siguiente

18. Pedirle a OpenCode el prompt de negocio (la aplicación puntual) 🟡 — *por proyecto*

Recién acá se describe qué va a hacer la aplicación en concreto: qué información guarda y qué pantallas tiene. Este prompt sí cambia de un proyecto a otro — es la parte que cada alumno completa según su propia idea.

- 18.1) Copiar y pegar, dentro de la misma sesión de OpenCode:

Necesito:

1. Una tabla en la base de datos (usá el MCP de Supabase ya conectado) con estos campos: CUIT, Nombre y Apellido, Domicilio, Teléfono, Correo Electrónico.
2. Un formulario para registrar un cliente nuevo, con esos mismos campos.
3. Una tabla debajo del formulario que liste todos los clientes ya cargados, mostrando todos esos campos.
4. Que se pueda editar y eliminar un cliente ya cargado desde la misma tabla.

Hacelo simple y funcional, con un diseño prolijo pero sin complicaciones innecesarias.

- 18.2) Esperar a que termine de crear la tabla vía MCP y armar las pantallas
- 18.3) Revisar el resultado directamente en el navegador

Con esto, OpenCode ya tiene todo lo que necesita para trabajar solo de acá en adelante. Las credenciales (Project URL y Publishable key) quedaron guardadas en el proyecto, y la conexión MCP queda activa — no hace falta volver a copiar ni pegar nada más. Cualquier pedido nuevo, por más estructural que sea (agregar una tabla, cambiar campos, relacionar información), se hace directamente en lenguaje natural: OpenCode ya sabe con qué credenciales trabajar y cómo modificar la base de datos.

⚠ ADVERTENCIA: el archivo `.env.local`, donde quedaron guardadas las credenciales, nunca debe subirse a GitHub — ahí vive información sensible de la cuenta. La buena noticia es que Next.js ya lo excluye automáticamente por defecto al crear el proyecto, así que en general no hay que hacer nada extra — pero vale la pena saber que este archivo en particular es distinto a todos los demás del proyecto.

Hasta acá, ya está construida la primera versión de la aplicación, funcionando en modo desarrollo, en la propia PC. A partir de acá empieza el ciclo de iteración: probar lo que se construyó, pedirle ajustes a OpenCode en lenguaje natural, probar de nuevo, guardar el avance, y repetir tantas veces como haga falta — exactamente como los retoques de la película, cambiando colores, fondos, comportamientos, hasta lograr el resultado deseado. Este ciclo (puntos 19 a 24) se repite libremente, todas las veces que se quiera, a lo largo de una o varias clases. Recién cuando la aplicación conforma tal como se la imaginó, se pasa a la etapa final: la publicación.

19. Abrir la Terminal de Windows y posicionarse en la carpeta del proyecto ● — *por cada iteración*

Cada vez que se retoma el trabajo sobre un proyecto ya existente, lo primero es “pararse” en la carpeta correcta, para que todo lo que se pida después aplique al proyecto correcto y no a otro.

- 19.1) Abrir la Terminal de Windows (o la terminal integrada de VS Code)
- 19.2) Escribir `cd` seguido de la ruta de la carpeta del proyecto

20. Lanzar OpenCode — *por cada iteración*

Es, en cierto sentido, el “encender la máquina” de cada sesión de trabajo: a partir de acá, todo lo que sigue se hace conversando con el agente.

- 20.1) Copiar y pegar en la consola: `opencode`

21. Levantar el modo desarrollo — *por cada iteración*

Se le pide a OpenCode que inicie el servidor local, para poder ver la aplicación funcionando en el navegador de la propia PC. Esta versión es privada: solo la puede ver quien está sentado frente a esa computadora en ese momento.

- 21.1) Pedirle a OpenCode: “Iniciá el servidor de desarrollo” (o, sin pasar por el agente, copiar y pegar directamente en la consola): `npm run dev`
- 21.2) Abrir el navegador y copiar y pegar en la barra de direcciones: `http://localhost:3000` (“localhost” significa literalmente “esta misma computadora”, no internet; por eso nadie más, desde ningún otro lugar, puede ver esta versión todavía)

22. Pedirle cambios a OpenCode — *por cada iteración*

Conviene siempre pedir cambios acotados, de a uno por vez, en vez de pedir todo un conjunto grande de cosas juntas — así es más fácil revisar que cada cambio funcionó como se esperaba. Si el cambio afecta la estructura de datos (por ejemplo, agregar un campo nuevo), OpenCode va a mostrar el plan antes de aplicarlo vía MCP — conviene revisarlo antes de confirmar.

- 22.1) Escribir el pedido concreto y preciso en el chat de OpenCode

23. Probar los cambios — *por cada iteración*

Antes de dar por bueno un cambio y guardarlo, hay que confirmar en la práctica que efectivamente funciona como se esperaba.

- 23.1) Volver al navegador y revisar que el cambio se comporte tal como se pidió

24. Pedirle a OpenCode que guarde los cambios en GitHub — *por cada iteración*

Esto deja registrado ese avance puntual en el historial completo del repositorio (equivalente técnico: `commit + push`, explicado más arriba).

- 24.1) Escribirle a OpenCode: “Guardá estos cambios en GitHub con un mensaje descriptivo”
- 24.2) (La primera vez que esto sucede, va a aparecer la ventana de login de GitHub descripta en el punto 5.4)

PARTE 3 — Publicación final en la web

25. Conectar el repositorio en Vercel y publicar por primera vez — *por proyecto*

A partir de este momento, Vercel toma el código guardado en el repositorio y genera con él una versión pública, real, accesible desde cualquier lugar, sin depender de que la PC del alumno esté prendida ni conectada a internet.

- 25.1) Ir a <https://vercel.com/dashboard> e iniciar sesión (si no lo está ya)
- 25.2) Click en “Add New...” → “Project”
- 25.3) Elegir el repositorio de GitHub correspondiente → “Import”
- 25.4) Dejar la configuración por defecto → “Deploy”
- 25.5) Esperar a que termine

Este primer despliegue va a quedar publicado, pero todavía sin poder conectarse a la base de datos — eso se resuelve en el siguiente paso.

26. Cargar las variables de entorno en Vercel — *por proyecto*

El archivo `.env.local`, donde OpenCode guardó las credenciales de Supabase, nunca se subió a GitHub — fue una decisión de seguridad tomada a propósito. Por eso Vercel, que solo puede ver lo que está en el repositorio, no tiene forma de acceder a esas credenciales por sí solo. Hay que cargarlas a mano, una sola vez, directamente en el panel de Vercel (un lugar de almacenamiento separado, no conectado al contenido del repositorio).

- 26.1) Dentro del proyecto en Vercel, ir a “Settings”
- 26.2) En el menú lateral, click en “Environments” (en algunas versiones de Vercel figura como “Environment Variables” directamente; es la misma sección)
- 26.3) Click en “Production”
- 26.4) Click en “Add Environment Variable”
- 26.5) Forma recomendada — cargarlas todas juntas: buscar la opción “Import .env” (u “importar desde archivo”) en esa misma pantalla, y pegar ahí el contenido completo del archivo `.env.local`. Esto carga todas las variables de una sola vez, sin margen de error de tipeo.
- 26.6) Alternativa — cargarlas una por una: por cada línea del `.env.local` (con formato `NOMBRE=valor`), completar un campo “Key” (el nombre, copiado exactamente igual, mayúsculas incluidas) y un campo “Value” (el valor correspondiente), y click en “Add Environment Variable” por cada una
- 26.7) Guardar
- 26.8) Vercel va a pedir un nuevo despliegue (“Redeploy”) para que la aplicación publicada empiece a usar estas variables — confirmar y esperar a que termine (1 a 3 minutos)

 **ADVERTENCIA:** cambiar o agregar variables de entorno no actualiza sola la versión ya publicada — siempre hace falta un nuevo “Redeploy” después, como en el paso 26.8. Si en el futuro se modifica alguna de estas credenciales, hay que repetir este mismo paso.

Sobre los links que entrega Vercel al finalizar: en la pantalla de detalles del despliegue van a aparecer varios links parecidos. El que hay que usar y compartir es el que aparece primero, con el formato `nombreproyecto-algo.vercel.app` (sin sufijos raros) — ese es el dominio de

producción estable. Los demás (con “git-main” o con un código largo) son variantes técnicas internas de Vercel, no hace falta compartirlas.

27. El despliegue en Vercel se actualiza solo — por cada iteración

Una vez que el proyecto ya quedó conectado a Vercel (punto 25) y con sus variables de entorno cargadas (punto 26), cada vez que se guardan cambios de código en GitHub, Vercel detecta automáticamente la novedad y publica sola la nueva versión, sin que nadie tenga que hacer nada más.

- 27.1) No requiere ninguna acción — alcanza con esperar 1 o 2 minutos y refrescar el link ya publicado

Identidades y claves a lo largo del proceso

1. Usuario de GitHub (por única vez)
2. Credencial de Git en la PC (por única vez)
3. Usuario de Supabase, vinculado a GitHub (por única vez)
4. Password del proyecto de Supabase (por proyecto)
5. Project URL de Supabase (por proyecto)
6. Publishable key de Supabase (por proyecto)
7. Conexión de la cuenta de OpenCode a Supabase, por OAuth (por única vez)
8. Conexión de OpenCode a Supabase vía MCP (acotada a un proyecto puntual, no a toda la cuenta)
9. Usuario de Vercel, vinculado a GitHub (por única vez)

Solución de problemas

“Aplicación de 16 bits no compatible” al ejecutar opencode

Esto significa que el ejecutable de OpenCode quedó corrupto durante la instalación — casi siempre porque se usó el comando corto (`npm install -g opencode-ai`, sin `--allow-scripts`) y Windows bloqueó, por seguridad, un paso interno necesario de la instalación. No es un problema de la PC ni de Windows: es que el programa quedó a medio instalar.

Para solucionarlo:

1. Desinstalar: `npm uninstall -g opencode-ai`
2. Revisar la carpeta `C:\Users\[usuario]\AppData\Roaming\npm\node_modules` y borrar a mano cualquier carpeta llamada `opencode-ai` o que empiece con un punto, si quedó alguna
3. Limpiar la caché de npm: `npm cache clean --force`
4. Cerrar la terminal por completo y abrir una nueva
5. Reinstalar con el comando correcto (ver punto 7 de este documento): `npm install -g --allow-scripts=opencode-ai opencode-ai`
6. Verificar: `opencode --version` (tiene que devolver un número de versión, sin ninguna ventana de error)

Al instalar OpenCode aparece un mensaje “npm warn install-scripts... blocked”

Es la misma causa del problema anterior, pero detectada a tiempo, antes de que el programa quede roto. Si aparece este mensaje, no hace falta desinstalar nada todavía — alcanza con instalarlo de nuevo, esta vez con el comando correcto: `npm install -g --allow-scripts=opencode-ai opencode-ai`

Al pedirle a OpenCode que se conecte al MCP, pide una contraseña de base de datos (DB_PASSWORD)

Esto quiere decir que el plugin de Supabase (punto 9 de este documento) no está instalado, y OpenCode está intentando armar una conexión directa a la base de datos en vez de usar el flujo oficial por MCP. Instalar el plugin (`opencode plugin opencode-supabase`, en una terminal, sin sesión de OpenCode abierta) y retomar la conexión desde el punto 16 de este documento.

Al escribir /supabase dentro de OpenCode, no pasa nada

El plugin de Supabase no está instalado todavía en ese proyecto. Ver el punto 9 de este documento.

Al ejecutar `opencode mcp auth supabase`, aparece un error de “no es una aplicación válida” o similar

Ese comando es de terminal, no del agente: no se le escribe al agente dentro de una sesión de OpenCode ya abierta. Hay que cerrar la sesión de OpenCode primero, y ejecutar el comando en una terminal común y corriente.

Notas de diseño pendientes

- Producir el esquema visual tipo rompecabezas (capas Frontend / Aplicación / Datos / Infraestructura)
- Producir el mapa de identidades y claves como pieza visual propia
- Verificar política de pausado de proyectos gratuitos en Supabase entre clases
- Documento aparte pendiente sobre conceptos avanzados de bases de datos (tablas, relaciones, integridad referencial)